MODUL PRAKTIKUM 7: SQL untuk Data Analyst Lanjutan (JOIN & Advanced Queries)

A. Tujuan Praktikum

1. Mahasiswa memahami cara menghubungkan Python dengan *database* berskala industri menggunakan **SQLAlchemy**.
2. Mahasiswa mampu menggabungkan banyak tabel relasional menggunakan `INNER JOIN`, `LEFT JOIN`, dan logika `RIGHT JOIN`.
3. Mahasiswa mampu membuat *Query* Bersarang (**Subqueries**) untuk penyaringan data bertingkat.
4. Mahasiswa mampu menggunakan **Window Functions** (seperti `RANK()`) untuk analisis analitik yang kompleks.

B. Persiapan Lingkungan Kerja (Environment)

Pada modul ini, kita akan menggunakan **SQLAlchemy**, sebuah *library* standar industri yang bisa menghubungkan Python ke hampir semua jenis *database* (MySQL, PostgreSQL, Oracle, SQLite). Buka Terminal/Command Prompt di VS Code dan jalankan perintah:

pip install pandas sqlalchemy

C. BAGIAN 1: Koneksi Database Menggunakan SQLAlchemy & Setup Tabel

Kita akan membuat dua tabel yang saling berhubungan (Relasional): Tabel Departemen dan Tabel Karyawan.

Praktik 1: Membuat Engine SQLAlchemy & Input Data Ketikkan dan jalankan kode ini (cukup satu kali) untuk menyiapkan data praktikum.

```python
import pandas as pd
from sqlalchemy import create_engine

# 1. Membuat Koneksi (Engine) menggunakan SQLAlchemy
# Format: 'jenis_db:///nama_file.db'
engine = create_engine('sqlite:///kantor.db')

# 2. Menyiapkan Data Dummy
data_dept = pd.DataFrame({
    'ID_Dept': ['D01', 'D02', 'D03', 'D04'],
    'Nama_Dept': ['IT', 'HR', 'Finance', 'Marketing']
})

data_karyawan = pd.DataFrame({
    'ID_Emp': [1, 2, 3, 4, 5],
    'Nama': ['Andi', 'Budi', 'Citra', 'Diana', 'Eka'],
    'Gaji': [700, 450, 800, 500, 900],
    'ID_Dept': ['D01', 'D02', 'D01', None, 'D03']
    # Perhatikan: Diana tidak punya Departemen (None), dan Marketing tidak punya karyawan
})

# 3. Export DataFrame Pandas langsung ke Database SQL (Otomatis membuat tabel)
data_dept.to_sql('Departemen', engine, if_exists='replace', index=False)
data_karyawan.to_sql('Karyawan', engine, if_exists='replace', index=False)

print("Database 'kantor.db' dan tabel relasional berhasil disiapkan dengan SQLAlchemy!")
```

D. BAGIAN 2: Menggabungkan Tabel (JOIN)

Dalam *database* relasional, data sering kali terpisah. Untuk menyatukannya, kita menggunakan **`JOIN`**.

Praktik 2: INNER JOIN, LEFT JOIN, dan Logika RIGHT JOIN

```
print("--- 1. INNER JOIN (Hanya data yang saling beririsan) ---")
# Menampilkan karyawan yang PASTI memiliki departemen
query_inner = """
    SELECT Karyawan.Nama, Karyawan.Gaji, Departemen.Nama_Dept
    FROM Karyawan
    INNER JOIN Departemen ON Karyawan.ID_Dept = Departemen.ID_Dept
"""
hasil_inner = pd.read_sql(query_inner, engine)
print(hasil_inner)
print("\n")

print("--- 2. LEFT JOIN (Semua Karyawan masuk, meski tak punya departemen) ---")
# Diana akan masuk di sini, dengan nama departemen bernilai NaN (Kosong)
query_left = """
    SELECT Karyawan.Nama, Karyawan.Gaji, Departemen.Nama_Dept
    FROM Karyawan
    LEFT JOIN Departemen ON Karyawan.ID_Dept = Departemen.ID_Dept
"""
hasil_left = pd.read_sql(query_left, engine)
print(hasil_left)
print("\n")

print("--- 3. RIGHT JOIN LOGIC (Semua Departemen masuk, meski tak ada karyawan) ---"
# Catatan: SQLite tidak memiliki perintah RIGHT JOIN asli.
# Triknya: Kita balik posisinya menjadi Departemen LEFT JOIN Karyawan
query_right = """
    SELECT Departemen.Nama_Dept, Karyawan.Nama
    FROM Departemen
    LEFT JOIN Karyawan ON Departemen.ID_Dept = Karyawan.ID_Dept
"""
hasil_right = pd.read_sql(query_right, engine)
print(hasil_right)
```

E. BAGIAN 3: Subqueries (Query Bersarang)

Subquery adalah *query* SQL di dalam *query* SQL. Biasanya digunakan untuk filter data yang dinamis. Misalnya: "Siapa karyawan yang gajinya di atas rata-rata perusahaan?" Kita tidak bisa menebak angka rata-ratanya secara manual.

Praktik 3: Memfilter Berdasarkan Hasil Perhitungan Lain

```
print("--- Subquery: Karyawan Bergaji di Atas Rata-rata ---")
query_sub = """
    SELECT Nama, Gaji
    FROM Karyawan
    WHERE Gaji > (
        SELECT AVG(Gaji) FROM Karyawan
    )
"""
hasil_sub = pd.read_sql(query_sub, engine)
print(hasil_sub)
```

F. BAGIAN 4: Window Functions (Fungsi Analitik)

Window Function sangat kuat untuk *Data Analyst*. Fungsi ini memungkinkan kita melakukan perhitungan agregasi (seperti membuat ranking) *tanpa* mengelompokkan/menghilangkan baris datanya (berbeda dengan **`GROUP BY`**).

Praktik 4: Membuat Sistem Ranking per Departemen

```
print("--- Window Function: Peringkat Gaji Tertinggi per Departemen ---")
# PARTITION BY: me-reset ranking setiap kali departemen berganti
# ORDER BY Gaji DESC: mengurutkan ranking berdasarkan gaji terbesar
query_window = """
    SELECT
        Nama,
        ID_Dept,
        Gaji,
        RANK() OVER (
            PARTITION BY ID_Dept
            ORDER BY Gaji DESC
        ) AS Peringkat_Gaji
    FROM Karyawan
    WHERE ID_Dept IS NOT NULL
"""
hasil_window = pd.read_sql(query_window, engine)
print(hasil_window)
```

G. TUGAS MANDIRI (Analisis Database E-Commerce)

```
import pandas as pd
from sqlalchemy import create_engine

engine_toko = create_engine('sqlite:///ecommerce.db')

df_pelanggan = pd.DataFrame({
    'ID_Pel': ['P1', 'P2', 'P3', 'P4'],
    'Nama_Pel': ['Fahmi', 'Gina', 'Hadi', 'Intan']
})

df_pesanan = pd.DataFrame({
    'ID_Order': [101, 102, 103, 104, 105],
    'ID_Pel': ['P1', 'P1', 'P3', 'P99', 'P2'], # P99 adalah pelanggan tak terdaftar
    'Total_Belanja': [250000, 150000, 400000, 50000, 800000]
})

df_pelanggan.to_sql('Pelanggan', engine_toko, if_exists='replace', index=False)
df_pesanan.to_sql('Pesanan', engine_toko, if_exists='replace', index=False)
```

Instruksi Pengerjaan (Gunakan SQL, eksekusi menggunakan SQLAlchemy):

1. (Skor 30) JOIN: Tulis *query* SQL (gunakan `LEFT JOIN`) untuk menampilkan *semua* nama pelanggan beserta `Total_Belanja`-nya. Jika ada pelanggan yang belum pernah belanja (seperti Intan), datanya harus tetap muncul (dengan nilai NaN/Null di belanjanya).
2. (Skor 35) Subquery: Tulis *query* SQL menggunakan *Subquery* untuk menampilkan detail Pesanan (`ID_Order` dan `Total_Belanja`) yang nilai belanjanya *lebih besar* dari rata-rata seluruh total belanja di tabel Pesanan.
3. (Skor 35) Window Function: Tulis *query* SQL menggunakan `RANK() OVER()` untuk membuat peringkat pesanan paling mahal (terbesar) secara global di dalam tabel Pesanan (tanpa `PARTITION BY`, hanya menggunakan `ORDER BY Total_Belanja DESC`). Tampilkan kolom `ID_Order`, `Total_Belanja`, dan `Peringkat`.